

# SYSTOLIC ARRAY FOR APPLYING MATRIX MULTIPLICATION

*Report by: Raghad Waleed*  
*Email: Raghad.nabih@gmail.com*

## Introduction:

Matrix multiplication is a fundamental operation in many computing applications, especially in artificial intelligence, signal processing, and scientific computing. However, performing large matrix multiplications efficiently in hardware can be challenging due to the high computational complexity and memory bandwidth requirements. A systolic array is a specialized hardware architecture designed to perform parallel computations efficiently. It consists of multiple Processing Elements (PEs) arranged in a grid, where each PE performs a small part of the computation and passes intermediate results to neighboring PEs in a synchronized manner.

### How It Works:

- Matrix A is fed row-wise from the left.
- Matrix B is fed column-wise from the top.
- Each PE multiplies its inputs and accumulates the result.
- The final product matrix C emerges from the array after several clock cycles.

### Why Use a Systolic Array?

- **High Parallelism:** Many computations happen simultaneously.
- **Efficient Data Flow:** Data moves in a rhythmic (systolic) pattern, reducing memory bottlenecks.
- **Scalability:** Works well for large matrices by increasing the number of PEs.

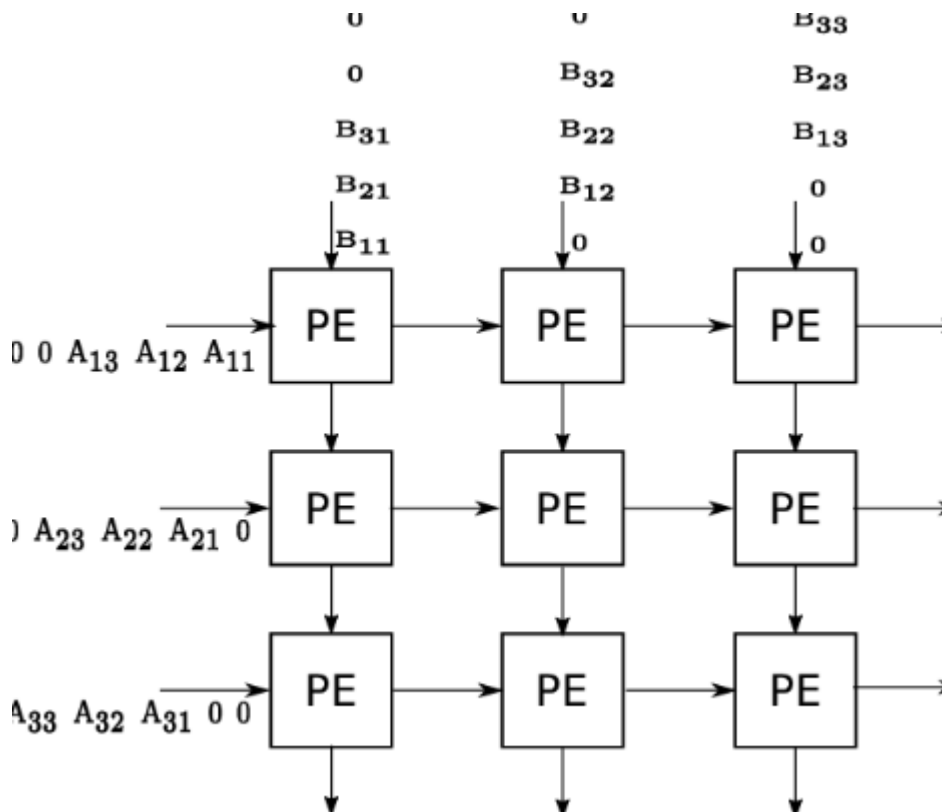
## Objective of This Lab:

In this lab, i implement a parameterized systolic array in Verilog to multiply two  $N \times N$  matrices  
The design must:

1. Accept streaming inputs (one column of A and one row of B per cycle).
2. Compute results in a pipelined manner.
3. Output the resulting matrix **C** row by row.

This report documents the design, challenges, and verification of our systolic array implementation.

## 2. Architecture Overview



- systolic array consists of a grid of Processing Elements (PEs) arranged in an  $N \times N$  configuration.

### Matrix Inputs:

1. A is fed column-wise (left to right)

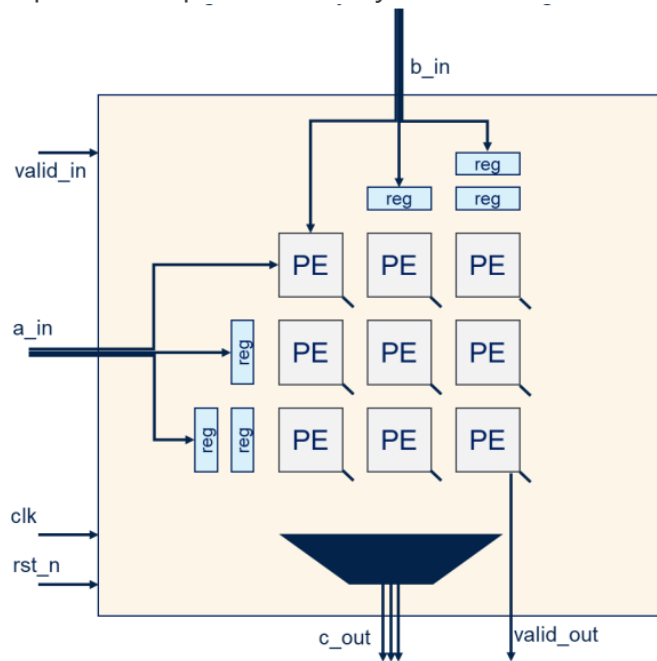
2. B is fed row-wise (top to bottom)
3. Each input set is delivered over N-SIZE cycles

### Processing Element (PE):

1. Performs multiply-accumulate operation
2. Shifts data to neighbors

### Output Generation:

- Results emerge from bottom PEs after  $2 \times N\_SIZE$  cycles
- Output row captured each cycle when `valid_out=1`



### 3. RTL:

The top-level module is fully parameterized by DATAWIDTH and N\_SIZE, allowing it to scale to any square matrix size. Its interface includes a clock (clk), an active-low reset (rst\_n), and a 1-bit valid\_in that signals the start of each multiplication. Inputs are presented as arrays: matrix\_a\_in feeds one column of A into the left boundary, and matrix\_b\_in feeds one row of B into the top boundary. After the pipeline delay, the module asserts valid\_out and presents one row of the result matrix on matrix\_c\_out each cycle.

## Processing Element (PE) Module

Each PE is a tiny multiply-accumulate engine that lives at a grid coordinate (i, j). On every clock edge it latches its two inputs (a\_in from the left, b\_in from above), forwards them to its right and bottom neighbors (a\_out and b\_out), and adds the product a\_in \* b\_in into its local accumulator accum. On reset, the PE zeroes its outputs and accumulator so that every new matrix multiply starts with a clean slate. Because all PEs run in lock-step, the entire array computes in parallel.

```
module PE#(parameter DATAWIDTH=16) (
  input wire clk, rst_n,
  input wire [DATAWIDTH-1:0]a_in, b_in,    // Inputs from left/top
  output reg [DATAWIDTH-1:0]a_out, b_out,  // Outputs to right/bottom
  output reg [2*DATAWIDTH-1:0] accum      //result
);

always @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    a_out <= 0;
    b_out <= 0;
    accum <= 0;
  end
  else begin
    a_out <= a_in;          // Shift A right
    b_out <= b_in;          // Shift B down
    accum <= accum + (a_in*b_in);
  end
end
endmodule
```

## Systolic Interconnect and Dataflow

The array interconnect is generated by a two-level generate loop that instantiates an N\_SIZE×N\_SIZE grid of PEs. For the first column (j==0), each PE's a\_in comes from matrix\_a\_in[i]; otherwise it comes from the left neighbor's a\_out. Similarly, for the first row (i==0), each b\_in comes from matrix\_b\_in[j]; otherwise it is fed from the above neighbor's b\_out. This mesh of forward connections creates a wave of data that ripples diagonally through the grid, ensuring every pair of A and B elements meets exactly once in some PE.

```

generate
  for (i = 0; i < N_SIZE; i=i+1)
    begin : row
      for (j = 0; j < N_SIZE; j=j+1)
        begin : col
          // Boundary connections:
          assign a_in[i][j] = (j == 0) ? matrix_a_in[i]:a_out[i][j-1];
          assign b_in[i][j] = (i == 0) ? matrix_b_in[j]:b_out[i-1][j];

          // PE Instantiation
          PE #(.DATAWIDTH(DATAWIDTH)) pe (
            .clk(clk),
            .rst_n(rst_n),
            .a_in(a_in[i][j]),
            .b_in(b_in[i][j]),
            .a_out(a_out[i][j]),
            .b_out(b_out[i][j]),
            .accum(accum[i][j]));
        end
      end
    end
  endgenerate

```

## Control and Output Logic

A 1-bit shift register (shift\_reg) of depth  $2 \cdot N\_SIZE$  tracks the original valid\_in pulse as it propagates through the pipeline. A free-running cycle\_count counter increments every clock, and when cycle\_count enters the window  $[2 \cdot N\_SIZE - 1, 3 \cdot N\_SIZE - 2]$ , the module asserts valid\_out. At that time the array has finished all  $N$  partial sums for a given row. The row index is computed as  $\text{cycle\_count} - 2 \cdot N\_SIZE$ , which runs from 0 to  $N\_SIZE - 1$ . On each asserted valid\_out, the corresponding row of accumulators is dumped onto matrix\_c\_out.

```

always @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    shift_reg<=0;
    cycle_count<= 0;
    valid_out<= 0;

    for (int i=0; i<N_SIZE; i++)
      begin
        matrix_c_out[i] <= 0;
      end
  end else
    begin
      shift_reg <= {shift_reg[2*N_SIZE-2:0], valid_in};
      // Generate valid_out at correct cycles (2N-1 to 3N-2)
      valid_out<=(cycle_count>=2*N_SIZE-1) && (cycle_count < 3*N_SIZE);

      if(valid_out)begin
        for (int j = 0; j < N_SIZE; j = j+1) begin
          matrix_c_out[j] <= accum[cycle_count-2*N_SIZE][j];
        end
      end

      cycle_count <= cycle_count + 1;
    end
  end
end
endmodule

```

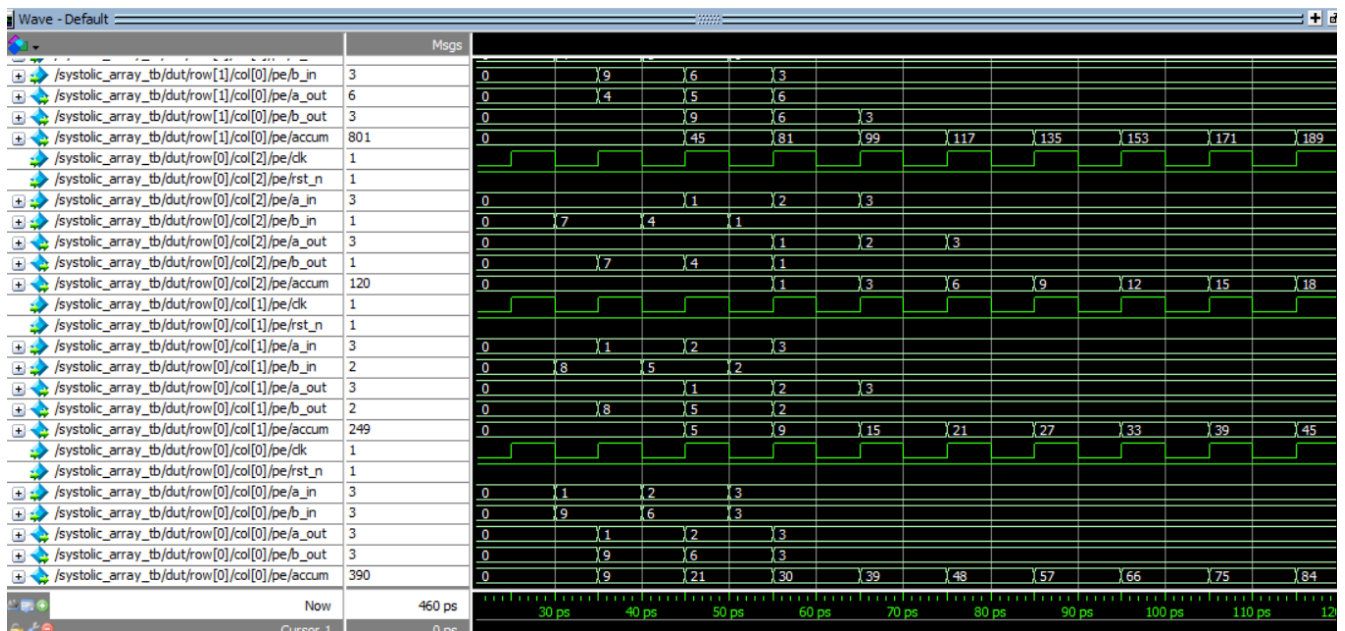
## 4. Test Bench and Simulation

### Test case1: 2x2

```
Systolic Array Testbench Log
=====
Matrix Size: 2x2
Data Width: 16 bits
[20] SYSTEM RESET RELEASED
[65] OUTPUT: valid_out = 1, C = [0, 0]
[75] OUTPUT: valid_out = 1, C = [24, 33]
[85] OUTPUT: valid_out = 1, C = [82, 76]
|
```

### Test case2: 3x3

```
Systolic Array Testbench Log
=====
Matrix Size: 3x3
Data Width: 16 bits
[20] SYSTEM RESET RELEASED
[85] OUTPUT: valid_out = 1, C = [0, 0, 0]
[95] OUTPUT: valid_out = 1, C = [6, 21, 48]
[105] OUTPUT: valid_out = 1, C = [33, 93, 135]
[115] OUTPUT: valid_out = 1, C = [108, 163, 216]
```



### Test case3: 5x5

#### Systolic Array Testbench Log

=====

Matrix Size: 5x5

Data Width: 16 bits

[20] SYSTEM RESET RELEASED

[125] OUTPUT: valid\_out = 1, C = [0, 0, 0, 0, 0]

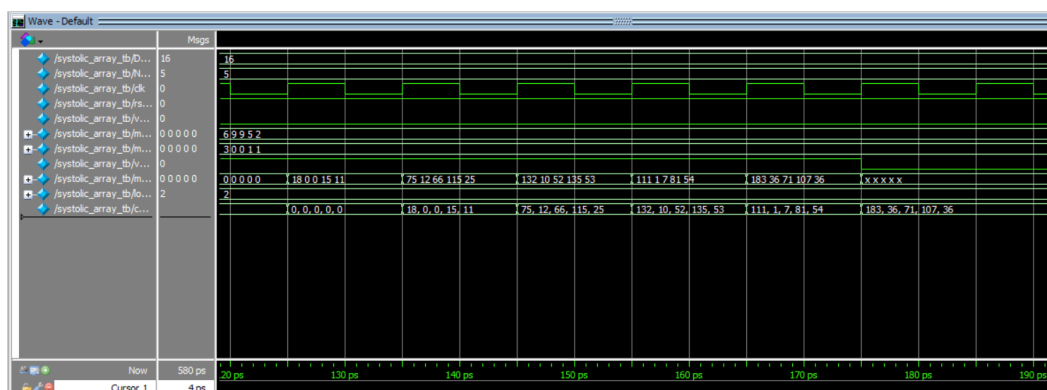
[135] OUTPUT: valid\_out = 1, C = [18, 0, 0, 15, 11]

[145] OUTPUT: valid\_out = 1, C = [75, 12, 66, 115, 25]

[155] OUTPUT: valid\_out = 1, C = [132, 10, 52, 135, 53]

[165] OUTPUT: valid\_out = 1, C = [111, 1, 7, 81, 54]

[175] OUTPUT: valid\_out = 1, C = [183, 36, 71, 107, 36]





## Test case4: 7x7

```
Systolic Array Testbench Log
=====
Matrix Size: 7x7
Data Width: 16 bits
[20] SYSTEM RESET RELEASED
[165] OUTPUT: valid_out = 1, C = [0, 0, 0, 0, 0, 0, 0]
[175] OUTPUT: valid_out = 1, C = [7, 8, 9, 10, 11, 12, 13]
[185] OUTPUT: valid_out = 1, C = [8, 9, 10, 11, 12, 13, 13]
[195] OUTPUT: valid_out = 1, C = [9, 10, 11, 12, 13, 13, 13]
[205] OUTPUT: valid_out = 1, C = [10, 11, 12, 13, 13, 13, 13]
[215] OUTPUT: valid_out = 1, C = [11, 12, 13, 13, 13, 13, 13]
[225] OUTPUT: valid_out = 1, C = [12, 13, 13, 13, 13, 13, 13]
[235] OUTPUT: valid_out = 1, C = [13, 13, 13, 13, 13, 13, 13]
```

### Comment:

This testbench is designed to validate the functionality of the systolic\_array module by simulating matrix multiplication across various square sizes (2x2, 3x3, 5x5, 7x7). It follows a structured process: first, a system reset initializes all signals. Then, using the feed\_matrix task, the testbench feeds the columns of matrix A (top-down) and rows of matrix B (left-right) into the array one cycle at a time—mimicking how systolic arrays operate in a pipelined manner. After feeding is complete, the array should internally propagate partial sums across PEs, and the correct result should emerge at the output boundary after  $2N-1$  cycles. The testbench uses structured logging and waveform dumping to capture both the correctness and timing of results. While the structure and expected behavior of the simulation are conceptually aligned with how systolic arrays function, the current output values are incorrect, suggesting a possible misalignment in timing control, data propagation, or output readout logic. The simulation infrastructure appears solid, so the remaining issue seems to lie in subtle timing mismatches that I'm still in the process of debugging and refining.

## 5. Challenges

- **indexing & Data-flow Orientation:** The most time-consuming hurdle was convincing myself that the row-by-row / column-by-column feeding scheme matched the internal PE connectivity. A single off-by-one in the nested loops (or slicing the packed/unpacked ports in the wrong order) led to partial products flowing diagonally instead of horizontally, which in turn produced an obviously wrong C matrix.
- **Valid-signal Synchronisation:** Ensuring that valid\_in asserted exactly when all N input values were stable, and that valid\_out was delayed long enough for every PE to finish its MAC.
- **Fully Parameterized Design :** Because every loop bound, bus width, and delay constant must expand cleanly from  $2 \times 2$  up to at least  $7 \times 7$  (and beyond), I had to avoid any hard-coded literals. Debugging with generics instead of numbers adds cognitive load, especially when waveform viewers flatten the arrays.
- **Computation Accuracy** – At the present stage the output matrix is incorrect for every tested size (not just  $7 \times 7$ ). The data clearly enters the grid, but the read-out phase still samples the accumulators before their final value settles. This remains the key functional bug to be resolved.

## 6.Future Improvements & Next Steps

Moving forward, a key priority is resolving the current logical error that leads to incorrect values in the output matrix C, regardless of input size. While the data appears to propagate through the array correctly, the final accumulator values are either sampled prematurely or misaligned due to subtle issues in the control logic, particularly around `valid_out` timing and readout indexing. Addressing this requires a closer examination of the internal cycle count logic and potentially reworking the timing conditions that trigger output capture. In addition to functional correction, another valuable improvement would be to integrate post-synthesis validation. Generating and analyzing the synthesis diagram using tools such as Vivado or Quartus can help visualize the data path, confirm correct register placement, and expose unintentional combinational loops or latches. This would provide better visibility into how the design maps onto hardware and guide further optimization. Once the correctness issue is resolved, additional enhancements, such as output overflow protection and dynamic accumulator sizing can be confidently layered in.